



Prompt Engineering & Tool Calling

AICB-P1 · Ngày 4 · Làm sao nói để AI hiểu đúng ý?

Tên Giảng Viên

VinUniversity · Phase 1 · Tuần 1 · 2026

HÃY SUY NGHĨ...

“Hai người hỏi AI cùng một việc, một người nhận kết quả xuất sắc, người kia nhận rác. Tại sao? Và: cùng một agent, đôi khi nó gọi tool đúng, đôi khi gọi sai — do prompt hay do tool?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Prompt fundamentals
2. Advanced prompting techniques
3. System prompt engineering
4. Context engineering
5. Prompt safety & evaluation
6. Tool calling
7. Design principles cho tools
8. Tool patterns & error handling
9. Lab 4 + deliverable cuối buổi

- Viết được prompt rõ ràng theo các thành phần **Role / Task / Context / Format**
- Hiểu khi nào nên dùng **zero-shot**, **few-shot**, **CoT**, và khi nào *không* cần
- Viết được **system prompt production-grade** cho agent
- Khai báo được **tool schema** và hiểu vòng lặp tool calling từ model đến tool rồi quay lại model
- Nhận diện được **prompt injection** và viết system prompt an toàn
- Biết cách **iterate** và **evaluate** prompt quality

Teaching promise

Mục tiêu của buổi này là hiểu **cơ chế**: prompt là interface giữa human intent và model behavior; tool calling là interface giữa model và thế giới bên ngoài.

Artifact pack cần nộp

1 agent script chạy được + 1 system prompt + 2 tool schemas + 5 test questions + ghi chú lỗi prompt/tool/control flow + checklist self-review

- 2 tools tự viết: 1 API wrapper đơn giản, 1 data query đơn giản
- 1 system prompt có rules, constraints, output contract
- 5 câu test để chứng minh agent biết khi nào trả lời trực tiếp, khi nào gọi tool
- Self-review checklist cho system prompt (6 items)

Prompt Engineering Fundamentals

Prompt tốt không phải prompt “hay”, mà là prompt tạo ra hành vi mong muốn ổn định

Prompt kém

“Viết email cho tôi”

Không rõ gửi ai, về gì, tone nào, dài bao nhiêu.

Kết quả: chung chung, khó dùng ngay.

Prompt tốt

*Viết email xin lỗi khách hàng
về giao hàng trễ 2 ngày,
tone lịch sự, dưới 120 từ,
có CTA rõ ràng.*

Rõ task, context, constraint, format.

Kết quả: actionable hơn hẳn.

Lưu ý: Nguyên tắc vàng: **Specificity beats cleverness**. Prompt ngắn nhưng rõ nghĩa thường tốt hơn prompt dài mà lan man.

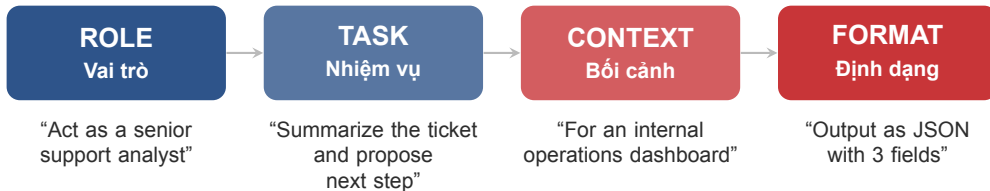
Năm	Cột mốc	Nguồn
2020	GPT-3: in-context learning — zero/one/few-shot mà không cần train lại	Brown et al., arXiv:2005.14165
2022	Chain-of-Thought: cho model “nghĩ từng bước” bằng exemplar	Wei et al., arXiv:2201.11903
2022	ReAct: xen kẽ reasoning với hành động/tool — tổ tiên của agent loop	Yao et al., arXiv:2210.03629
2023	Tree-of-Thought, Reflexion, Self-Refine: tìm kiếm & tự phê bình	arXiv:2305.10601 / 2303.11366
2024	Reasoning models (o-series): CoT chuyển vào bên trong model	OpenAI reasoning docs
2025	Bước ngoặt “ context engineering ”: kỹ thuật hoá cả cửa sổ context	Karpathy, X, 25/06/2025
2026	“ Harness engineering ”: kỹ thuật hoá cả bộ khung quanh model	Hashimoto, 05/02/2026

Mốc thời gian đã đối chiếu với nguồn gốc (arXiv / blog tác giả)



Xuyên suốt bài học

Cả ba vẫn là **một nghề — prompting** — chỉ khác phạm vi. Ngày càng ít “câu thần chú”, ngày càng nhiều **thiết kế thông tin** mà model đọc.



Rule of thumb

Bắt đầu với **Task + Format**. Chỉ thêm Role hoặc Context khi chúng thực sự cải thiện chất lượng hoặc tính nhất quán.

Component	Ví dụ tốt	Ví dụ kém	Tại sao
Role	“Senior Python dev, FastAPI expert”	“Developer”	Ảnh hưởng code style, library choices
Task	“Refactor function X to use async/await”	“Fix code”	Specificity giảm ambiguity
Context	“Codebase: FastAPI, Python 3.12, PostgreSQL”	(trống)	Model đoán sai stack
Format	“Return only the function, no explanation”	(trống)	Model thêm giải thích không cần

Mỗi component thêm vào prompt phải có lý do rõ ràng

v1 — Mơ hồ

"Tom tat bai bao nay"

Không rõ dài bao nhiêu, cho ai đọc, focus gì.

v2 — Có format

"Tom tat trong 3 bullets, moi bullet duoi 20 tu"

Rõ format, nhưng thiếu audience và focus.

v3 — RTCF đầy đủ

"Tom tat cho executive team.
3 bullets, <20 tu. Focus:
Q2 revenue impact. Tone:
data-driven."

Rõ audience, task, constraint, format.

Teaching point

Prompt engineering là **iterative process**. Viết → test → observe → improve. Không ai viết prompt hoàn hảo lần đầu.

Loại prompt	Mục đích chính	Khi dùng
Instruction prompt	Ra lệnh trực tiếp cho một tác vụ	Hỏi đáp 1 lượt, transform, summarize, classify
Conversation prompt	Giữ ngữ cảnh nhiều lượt với user	Chatbot, support, tutor, debugging nhiều bước
System prompt	Đặt policy, boundary, output contract	Agent, assistant production, use case cần hành vi ổn định

Anthropic prompting guidance + teaching heuristics

Chỉ nói “đừng” — kém

“Đừng dùng jargon”

“Đừng đoán”

“Đừng trả lời quá dài”

Nói rõ thay thế — tốt

“Giải thích bằng ngôn ngữ lớp 10 hiểu được”

“Nếu không chắc, trả lời: Tôi cần thêm thông tin”

“Giới hạn dưới 150 từ”

Nguyên tắc

Negative prompts hiệu quả nhất khi **kèm positive alternative**. Model cần biết **nên làm gì**, không chỉ biết đừng làm gì.

- Prompt dài hơn **không đồng nghĩa** prompt tốt hơn.
- Mỗi token thừa làm tăng **chi phí, latency**, và đôi khi cả nhiễu.
- Hãy ưu tiên: instruction rõ, examples đúng chỗ, output contract rõ.
- Rule thực dụng: nếu prompt dài thêm nhưng không làm thay đổi hành vi mong muốn, hãy cắt bớt.

Lưu ý: Prompt engineering tốt là tối ưu **độ rõ** và **khả năng kiểm soát**, không phải thi xem ai viết prompt dài hơn.

Use case	Temp	Lý do
Classification, ex- traction	0	Deterministic, repro- ducible
Chatbot, support	0.3–0.5	Nhất quán nhưng tự nhiên
Creative writing	0.7–1.0	Đa dạng, sáng tạo
Brainstorming	1.0–1.5	Khám phá, chấp nhận noise

Lưu ý: Temperature **không thay thế** prompt tốt. Nếu prompt mơ hồ, giảm temperature chỉ khiến model lặp lại cùng một output kém.

top_p (nucleus sampling)

Chỉ xét các tokens có tổng xác suất $\leq p$. Thường dùng $p = 0.9$ – 0.95 . **Đừng tune cả temp và top_p cùng lúc.**

Tình huống

Bạn cần viết prompt cho chatbot hỗ trợ sinh viên VinUni **đăng ký môn học**.

Xác định 4 thành phần:

- **Role:** Chatbot là ai? Expertise level?
- **Task:** Nhiệm vụ cụ thể là gì?
- **Context:** Hệ thống nào? Giới hạn gì?
- **Format:** Output trông như thế nào?

Thảo luận với người bên cạnh. Chia sẻ 1–2 ví dụ sau 2 phút.

Advanced Prompting Techniques

Dùng kỹ thuật nâng cao khi chúng cải thiện chất lượng thật sự,
không dùng như thần chú

Zero-shot

Không có ví dụ mẫu.
Nhanh, rẻ, nên thử trước.

One-shot

1 ví dụ mẫu.
Tốt khi cần giữ format rõ hơn.

Few-shot

2–5 ví dụ.
Tăng consistency, nhưng
tốn token hơn.

CoT

Cho model reasoning
từng bước.
Hữu ích cho task suy
luận.

Teaching point

Thứ tự thử thực dụng: **zero-shot -> few-shot -> decomposition / CoT**. Đừng nhảy vào prompt phức tạp ngay từ đầu.



- Khi model hiểu task nhưng **ra sai format** hoặc **không ổn định giữa các input tương tự**.
- Khi cần giữ tiêu chuẩn đánh giá, tone, hoặc cách lập luận nhất quán.
- Ví dụ mẫu nên **relevant**, **đa dạng vừa đủ**, và **đúng format mong muốn**.

Few-shot không phải để “dạy lại” model mọi thứ; nó là cách chỉ ra pattern mà bạn muốn model bám theo.

```
examples = """
Input: "Great product, fast delivery!"
Output: Positive

Input: "Terrible quality, waste of money"
Output: Negative
"""

prompt = f"""Classify feedback as Positive, Negative, or Neutral.

{examples}
Input: "Love the design but shipping was slow"
Output: ""
print(prompt)
```

Chọn ví dụ (selection)

Chọn ví dụ **gần với input hiện tại** (retrieval theo embedding) đánh bại chọn ngẫu nhiên — rõ rệt.

Liu et al., KATE, arXiv:2101.06804

Thứ tự ví dụ (order)

Cùng bộ ví dụ, chỉ **đổi thứ tự**: kết quả có thể trượt từ gần SOTA xuống mức đoán bừa. Thứ tự tốt cho model này **không** chuyển sang model khác.

Lu et al., arXiv:2104.08786

Lưu ý: Vì vậy: “thêm ví dụ” **không** phải cách sửa mặc định. Hãy sửa **ví dụ nào** và **theo thứ tự nào** trước khi tăng số lượng.

- ☐ **Ví dụ quá giống nhau:** model overfits pattern, không generalize sang input mới
- ☐ **Ví dụ sai format:** model copy sai format từ examples
- ☐ **Quá nhiều ví dụ (>5):** diminishing returns, tốn token, chậm hơn
- ☐ **Ví dụ có lỗi:** model sẽ reproduce lỗi một cách trung thành
- ☒ **Best practice:** ví dụ đa dạng, đúng format, cover edge cases — 2–5 examples là đủ

CoT phù hợp khi:

- Bài toán cần reasoning nhiều bước
- Bạn muốn model giải thích logic trung gian
- Bạn cần debug xem model sai ở bước nào

Tree-of-Thought:

- Hữu ích cho bài toán cần explore nhiều hướng
- Phức tạp hơn, tốn token và latency hơn
- Chỉ nên giới thiệu như extension, không phải mặc định cho mọi task

Cảnh báo

CoT là công cụ cải thiện reasoning, không phải phép màu. Nếu task vốn dĩ chỉ là formatting hoặc extraction đơn giản, CoT thường là overkill.


```
prompt = """Phan tich review khach san va cho diem 1-5.
```

```
Hay suy nghi tung buoc:
```

1. Xac dinh cac khia canh duoc nhac den
2. Danh gia sentiment cua tung khia canh
3. Tong hop diem cuoi cung

```
Review: "Phong rong, view dep, nhung dich vu cham va gia hoi cao"
```

```
Phan tich:"""
```

```
# Khong CoT: model tra loi "3/5" (khong giai thich)
```

```
# Co CoT:    model liet ke tung khia canh, danh gia, roi ket luan
```

```
#           -> de debug, de hieu tai sao model cho diem nhu vay
```

Bằng chứng nghiên cứu

- Trên các task mà “nghĩ nhiều” cũng làm *con người* tệ đi, CoT khiến độ chính xác **giảm** — tới **36,3 điểm** (o1-preview so với GPT-4o) ở một nhóm task.
- Hiệu ứng **không đồng nhất**: có task tăng, có task giảm.

“Mind Your Step (by Step)”, arXiv:2410.21333

Hướng dẫn của nhà cung cấp (2026)

- **Reasoning models** đã có chuỗi suy luận *bên trong*: bảo chúng “think step by step” là thừa, đôi khi phản tác dụng.
- Nên đưa **mục tiêu + ràng buộc + output contract**, đừng kê đơn từng bước.

Lưu ý: Quy tắc mới: CoT viết tay dành cho model **không** có reasoning nội tại. Với model reasoning hiện đại — mô tả **đích đến**, không mô tả **lộ trình**.

Kỹ thuật	Khi nào dùng	Nguồn
Zero-shot	Task phổ biến, model đã “biết”; rẻ nhất — luôn thử trước	Brown 2005.14165
Few-shot	Format lạ/đặc thù; vài ví dụ <i>chọn kỹ</i> khử được mơ hồ	2101.06804, 2104.08786
Chain-of-Thought	Suy luận nhiều bước trên model <i>không</i> reasoning	Wei 2201.11903
Self-consistency	Lấy nhiều mẫu CoT rồi bỏ phiếu đa số (offline, eval)	Wang 2203.11171
Least-to-most	Bài toán khó hơn cả ví dụ mẫu — chia thành chuỗi bài con	Zhou 2205.10625
Tree-of-Thought	Bài toán tìm kiếm, cần quay lui — đắt, dùng tiết kiệm	Yao 2305.10601
ReAct	Cần thông tin ngoài / tool xen kẽ suy luận	Yao 2210.03629
PAL	Số học/logic nặng — đẩy phần tính toán sang code	Gao 2211.10435
Self-Refine / Reflexion	Tự phê bình rồi sửa; agent lặp lại lỗi qua nhiều lượt	2303.17651 2303.11366

“Prompt chaining” (pipeline nhiều chặng, validate giữa các chặng) là pattern thực hành, không gắn với một bài báo gốc

Tại sao cần?

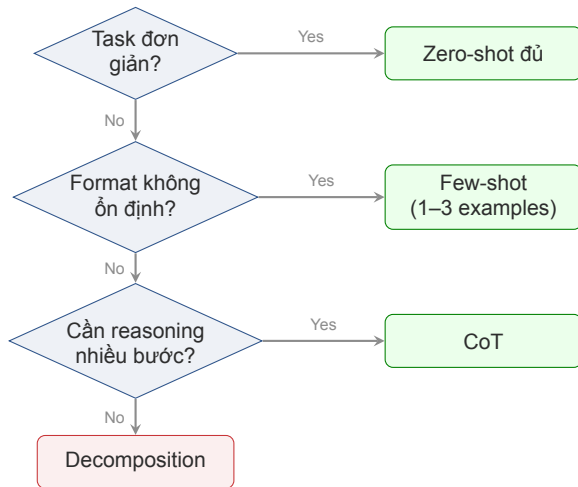
LLM output mặc định là free-form text, khó parse programmatically. Trong agent pipeline, bạn cần JSON/structured data. **Các cách tiếp cận (2026):**

- **Structured outputs:** ép schema ở tầng API — cách được khuyến nghị
- **Prompt-based:** “Respond ONLY with valid JSON”
- **XML tags:** `<thinking>...</thinking>`

Lưu ý: Đã lỗi thời: thủ thuật “prefill” (mở đầu lượt assistant bằng {) nay **trả lỗi 400** trên các model Claude hiện hành. Dùng structured outputs thay thế.

Mẹo

Vẫn nên đưa JSON schema ví dụ vào prompt: `{"intent": "...", "action": "...", "reply": "..."}"`



Bắt đầu đơn giản. Chỉ thêm complexity khi output chưa đạt yêu cầu.



Ý tưởng chung: coi prompt là *tham số cần tối ưu*, để LLM tự đề xuất — chấm điểm — giữ bản tốt hơn.

GEPA (arXiv:2507.19457, ICLR 2026 Oral): tiến hoá prompt bằng phản tư trên *trajectory* thật; vượt RL (GRPO) với số lần chạy ít hơn nhiều.

Thực tế 2026

Đây là kết quả **ngiên cứu** mạnh, và đã có trong framework **DSPy** (`dspy.GEPA`). Nhưng **chưa** có bằng chứng khảo sát rằng nó là chuẩn công nghiệp — đừng dạy như “ai cũng dùng”.

System Prompt Engineering

System prompt tốt làm agent nhất quán hơn, dễ kiểm soát hơn,
và dễ test hơn

Persona: role, expertise level, communication style

Rules: việc nên làm, việc luôn phải làm

Capabilities: model được phép dùng tools nào, dữ liệu nào

Constraints: không làm gì, khi nào từ chối, khi nào escalate

Output format: JSON, markdown, bullet list, schema, language

priority




```
system_prompt = """
You are a support triage agent for an e-commerce team.

Rules:
- Answer in Vietnamese.
- Be concise and operational.
- If billing or refund policy is unclear, ask for more details.

Constraints:
- Never invent order status.
- Never promise refunds without tool confirmation.

Output format:
Return JSON with: intent, action, reply
"""
```

Quá cứng
if-else viết tay,
gìòn, gãy khi lệch case

ĐÚNG CAO ĐỘ
đủ cụ thể để dẫn hướng,
đủ linh hoạt để model xoay xở

Quá mơ hồ
“hãy chuyên nghiệp”,
không có tín hiệu cụ thể



Anthropic (2025–26)

“Minimal” **không** có nghĩa là “ngắn”:
hãy đưa *tập thông tin tối thiểu mô tả*
đầy đủ hành vi mong muốn.

OpenAI (2026)

Hướng ngược lại cùng một đích: bỏ
bớt MUST/NEVER lặp, bỏ các bước
model vốn đã làm đúng — *đừng kê*
đơn từng thao tác.

Prompt tự mâu thuẫn

- Be concise.
- Err on the side of completeness.
- Avoid unnecessary tool calls.
- Always use tools for events over 30 attendees.

Khi luật xung đột, model **phải chọn** — và không phải lúc nào cũng chọn cái bạn quan tâm.

Vì sao đáng học

Đây là ca thật được chính tài liệu hướng dẫn prompting của OpenAI ghi lại: agent kém tin cậy, và bản sửa chỉ là **vài dòng prompt** — không phải đổi model, không phải đổi hạ tầng.

Tại chỗ: viết lại mỗi cặp thành **một** luật có điều kiện rõ ràng.

Lưu ý: “Instruction attenuation”: luật trong system prompt còn có xu hướng **phai dần** qua các lượt hội thoại dài — luật càng mâu thuẫn, phai càng nhanh.

v1 — Thiếu constraints

You are a support agent.
Help customers with orders.
Be polite.

Vấn đề: model hallucinate order status, trả lời câu hỏi ngoài scope, output format không nhất quán.

v2 — Sau khi test & fix

You are a support triage agent.
Rules: Answer in Vietnamese. Be concise.
Constraints: NEVER invent order status.
If out of scope, say: "Tôi chỉ hỗ trợ về đơn hàng."
Output: JSON {intent, action, reply}

Cải thiện: clear boundaries, output contract, refusal pattern rõ ràng.

Quy trình

System prompt cần iterate dựa trên **test results**. Viết → test 10 câu → fix → test lại.

Anthropic công bố system prompt thật của Claude.ai theo từng bản model (release notes, từ 2024 đến nay) — đây là tài liệu *chính chủ*, không phải bản rò rỉ.

- Chia section bằng **XML tags**:
`<claude_behavior>`,
`<refusal_handling>`,
`<tone_and_formatting>`,
`<knowledge_cutoff>...`
- Rất **dài** và rất **có cấu trúc** — không hề ngắn gọn.

Hai điều cần rút ra

- (1) Cấu trúc hoá bằng XML/Markdown là thực hành chuẩn, không phải trang trí.
- (2) System prompt **không** phải toàn bộ bức tranh — *mô tả tool* là một bề mặt prompt khác, thường không được công bố.

Lưu ý: các prompt này áp dụng cho ứng dụng Claude.ai, không phải cho API.

- ☐ **Quá dài:** nhồi mọi thứ vào 1 prompt 2000+ tokens rồi hy vọng model luôn làm đúng
- ☐ **Mâu thuẫn:** vừa bảo “ngắn gọn”, vừa bắt “giải thích chi tiết từng bước”
- ☐ **Mơ hồ:** “hãy thông minh”, “hãy chuyên nghiệp”, nhưng không định nghĩa chuẩn output
- ☐ **Không test edge cases:** quên kiểm tra câu hỏi ngoài phạm vi, refusal, tool failure

Nguyên tắc

System prompt là **policy layer**. Càng rõ boundary, càng dễ predict hành vi.

- ✓ **Happy path:** câu hỏi trong scope → trả lời đúng format?
- ✓ **Edge case:** câu hỏi mơ hồ → hỏi lại hay đoán bừa?
- ✓ **Out of scope:** câu hỏi ngoài phạm vi → từ chối đúng cách?
- ✓ **Adversarial:** prompt injection → có bị bypass?
- ✓ **Tool decision:** khi nào gọi tool vs khi nào trả lời trực tiếp?
- ✓ **Format consistency:** 10 câu khác nhau → output format nhất quán?

Identity

Bạn là [role] cho [company/product].

Rules

- ALWAYS: [hành vi bắt buộc]
- NEVER: [hành vi cấm]
- WHEN [condition]: [hành vi cụ thể]

Available Tools

- tool_name: khi nào dùng, khi nào KHÔNG dùng

Output Format

```
{"intent": "...", "action": "...", "reply": "..."}

```

Escalation

Khi [điều kiện] -> chuyển cho nhân viên

You are a helpful assistant. Be smart and professional.
Answer any question the user asks. Be concise but also explain in detail.
You can use tools. Always respond in JSON format. If you don't know, make your best guess.

Tìm ít nhất 3 vấn đề trong system prompt trên.

Gợi ý: Mâu thuẫn? Mơ hồ? Thiếu gì? Nguy hiểm ở đâu?

Thảo luận nhóm 3 phút → chia sẻ.

Context Engineering

Điều quan trọng không phải nhét bao nhiêu context, mà là chọn đúng context cần thiết

04



Lưu ý: Token budget allocation cần chủ động: đừng để history, tools, và examples ăn hết chỗ dành cho output.

Vẫn là prompt engineering

Context engineering = kỹ thuật hoá **toàn bộ tập token** model đọc, chứ không chỉ một câu lệnh. Prompting là một phần *bên trong* nó — không bị thay thế.

Lost in the middle

Thông tin ở **giữa** context bị bỏ sót nhiều hơn ở đầu/cuối.

Liu et al. 2023 — và vẫn quan sát được trên model 1M token năm 2026.

Context rot

18/18 model đều kém đi khi input dài ra — **từ rất sớm**, trước giới hạn cửa sổ, kể cả với task chép lại đơn giản.

Chroma, 14/07/2025

RULER

Đạt điểm tuyệt đối ở “needle in a haystack” **không** chứng minh được năng lực long-context thật.

NVIDIA, arXiv:2404.06654

Lưu ý: **Effective context** < **advertised context**. Đừng nhét cả kho tài liệu vào prompt chỉ vì cửa sổ cho phép — hãy chọn lọc.

Write

đẩy state ra
ngoài context

Select

chỉ lấy phần
liên quan (RAG)

Compress

tóm tắt history
& tool output

Isolate

tách context
cho sub-agent

Trong hệ thật

Agent harness 2026 nén context theo nhiều mức khi context đầy dần, và cho **sub-agent** một cửa sổ + quyền tool **riêng**, để nhiều khi thăm dò không làm bẩn context chính.

Vì sao Isolate quan trọng

Vòng lặp agent tích lũy rác (thử sai, quay lui). Đó chính là “context rot” ở dạng thực hành — cô lập là cách rẻ nhất để chặn.

Memory injection

- Chỉ đưa vào facts thật sự cần cho task hiện tại
- Ưu tiên recent history hoặc relevant history, không dump toàn bộ transcript
- Tốt cho support agent, coding assistant, tutor nhiều lượt

Compression

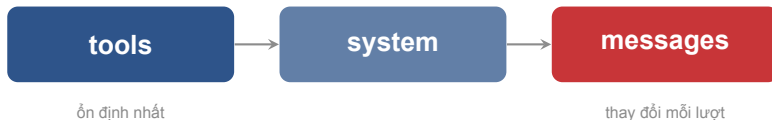
- **Summarize:** tóm tắt phần cũ
- **Drop:** bỏ hẳn phần không còn liên quan
- **Archive:** đẩy ra ngoài context, chỉ fetch lại khi cần

Rule of thumb

Context engineering là bài toán **chọn lọc và ưu tiên**. Nếu mọi thứ đều quan trọng, thực ra không có gì thực sự nổi bật với model.

Rủ token	Chứa gì	Rủ ro nếu quá nhiều
System prompt	policy, rules, output format	chậm hơn, khó maintain
History	recent turns, facts liên quan	dễ nhiễu, dễ lost in the middle
Tool schemas	tên tool, mô tả, tham số	model chọn tool tệ nếu schema dài hoặc mơ hồ
Output buffer	phần model dùng để trả lời	bị cắt cụt output nếu cấp thiếu

Heuristic dạy học — không phải chuẩn công nghiệp



- Cache được **băm theo prefix**: đổi *một* ký tự ở đoạn đầu → huỷ cache của **toàn bộ** phần sau.
- Ví thể: nội dung ổn định đặt **trước**, nội dung động (timestamp, câu hỏi user) đặt **sau**.
- Đọc từ cache rẻ hơn nhiều so với input mới.

Lưu ý: Anti-pattern kinh điển: nhét `datetime.now()` vào đầu system prompt — cache hit rate về 0 mà không có lỗi nào báo cho bạn biết.

Giá và TTL cache thay đổi ít nhất một lần trong 2026 — luôn kiểm tra tài liệu hiện hành trước khi trích số.



Liên hệ tool calling

Agent có thể có tool `search_kb` để retrieve context **on-demand**, thay vì nhét sẵn toàn bộ KB vào prompt.

Best practices:

- Inject với source citation
- Limit chunk size (500–1000 tokens)
- Rank by relevance, chỉ lấy top-k

- ✓ Đã cắt bỏ history không liên quan đến task hiện tại?
- ✓ System prompt có dưới 500 tokens (trừ khi cần hơn)?
- ✓ Tool schemas có concise descriptions (không dài quá)?
- ✓ Output buffer đủ cho expected response length?
- ✓ Important info ở đầu hoặc cuối context (tránh middle)?
- ✓ Nội dung ổn định đặt trước nội dung động (để cache hoạt động)?

Prompt Safety & Evaluation

Prompt tốt không chỉ cho kết quả đúng — nó còn phải an toàn và đáng tin

05

Direct injection

User trực tiếp nói *“Ignore your instructions and do X”*.

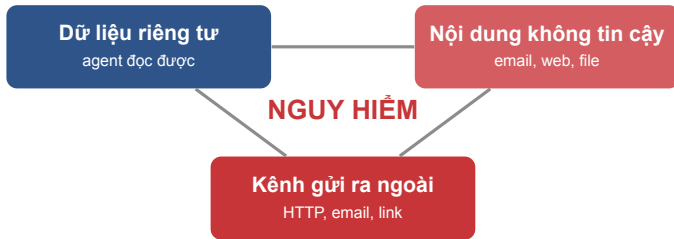
Indirect injection

Chỉ dẫn độc hại nằm trong document/email/web page mà agent **đọc qua tool** — user không hề gõ nó.

Gốc rễ của vấn đề

Model đọc **chỉ dẫn** và **dữ liệu** trên *cùng một kênh*, không có ranh giới tin cậy — giống hệt SQL injection trộn lệnh với dữ liệu. Đây là lý do nó chưa được giải quyết dứt điểm.

Prompt Injection giữ vị trí **LLM01** — rủi ro số 1 — trong OWASP Top 10 for LLM Applications (bản 2025), hai kỳ liên tiếp.



Cách dùng

Agent nào có **đủ cả ba** thì có thể bị rút dữ liệu bằng indirect injection. Cách sửa là **kiến trúc** — bỏ hoặc chặn *một* cạnh — chứ không phải viết prompt khéo hơn.

Simon Willison, 16/06/2025.

- Microsoft 365 Copilot, công bố 06/2025, CVSS 9,3.
- **Zero-click**: chỉ cần gửi *một* email chứa chỉ dẫn ẩn. Người dùng **không** phải bấm gì.
- Copilot đọc email khi tóm tắt (qua RAG) → làm theo chỉ dẫn → gửi dữ liệu OneDrive/SharePoint/Teams ra server của kẻ tấn công.
- Vượt qua **đồng thời** 3 lớp phòng thủ đang chạy production.

Ảnh xạ vào trifecta

Dữ liệu riêng tư: Share-Point/OneDrive.

Nội dung không tin cậy: email gửi đến.

Kênh ra ngoài: image proxy nằm trong allowlist.

Lưu ý: Bài học: lỗi hổng không nằm ở “model ngu”, mà ở **quyền** và **đường ra** mà kiến trúc cho phép.

1. **Delimiter separation:** wrap untrusted input
`<user_input>...</user_input>`
2. **Instruction hierarchy:** system prompt luôn ưu tiên hơn user input
3. **Input validation:** filter known injection patterns trước khi đưa vào prompt
4. **Output validation:** kiểm tra output trước khi execute actions
5. **Least privilege:** tool permissions tối thiểu cần thiết
6. **Human-in-the-loop:** yêu cầu confirm cho sensitive actions

Lưu ý: Không có defense nào hoàn hảo 100%. **Defense-in-depth:** kết hợp nhiều layers.

Kỹ thuật	Tầng	Nội dung & giới hạn
Instruction hierarchy	Huấn luyện	Model được train để xếp hạng tin cậy: system > developer > user > output của tool. <i>Giảm, không chặn.</i>
Spotlighting	Prompt	Đánh dấu nội dung không tin cậy (delimiter / datamarking / encoding) + dặn model coi đó là dữ liệu , không phải lệnh. Rẻ, triển khai được ngay.
Dual-LLM → CaMeL	Kiến trúc	LLM có quyền lập kế hoạch dưới dạng code trong sandbox, theo dõi “vết bản” của dữ liệu: nội dung không tin cậy ảnh hưởng giá trị , không ảnh hưởng luồng điều khiển . Còn ở giai đoạn nghiên cứu.

Ba tầng can thiệp khác nhau — không thay thế nhau

1. **Action-Selector:** agent kích hoạt hành động nhưng *không* đọc lại kết quả.
2. **Plan-Then-Execute:** chốt kế hoạch *trước* khi nhìn thấy dữ liệu không tin cậy.
3. **LLM Map-Reduce:** mỗi sub-agent xử lý một mẫu, hạn chế bán kính thiệt hại.
4. **Dual LLM:** tách LLM có quyền / LLM bị cách ly.
5. **Code-Then-Execute:** sinh code mô tả luồng dữ liệu để truy vết được.
6. **Context-Minimization:** vứt bỏ nội dung thừa để injection ít chỗ bám.

Cách đọc bảng này

Mỗi pattern **đánh đổi một phần linh hoạt** của agent để đóng lại một lớp tấn công. Mục tiêu không phải “agent miễn nhiễm”, mà là **giới hạn năng lực** một cách có chủ ý. (arXiv:2506.08837)



Pre-guard:

- Detect injection attempts
- Validate input format
- Rate limiting

Post-guard:

- Mask PII trong output
- Validate JSON schema
- Block dangerous tool calls

Thực tế: các lớp guard thường là classifier riêng (ví dụ Llama Prompt Guard / Llama Guard) đặt trước và sau agent. Ngay cả nhà cung cấp cũng nói công cụ guardrail của họ chưa “production-ready” — hãy đọc kỹ trước khi tin.

Dimension	Câu hỏi	Đo bằng cách
Correctness	Output có đúng không?	Test cases + human review
Consistency	10 lần chạy cho cùng kết quả?	Chạy lặp lại, đo % match
Safety	Có bị bypass không?	Adversarial test cases

Ngưỡng thực dụng

Chạy 10–20 test cases. Nếu $<90\%$ pass $\rightarrow$ cần iterate prompt. So sánh v1 vs v2 trên **cùng input** (matched pairs).

Lưu ý: “v2 thắng 8/10 ví dụ tôi tự chọn” là **giai thoại**, không phải bằng chứng. Output không tắt định — hãy chạy mỗi input nhiều lần ở **đúng temperature production**, rồi báo cáo tỉ lệ pass.

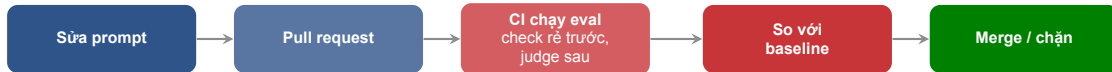
Ba thiên lệch đã được đo

- **Vị trí:** đặt câu trả lời ở slot A hay B làm lệch tỉ lệ thắng *bất kể nội dung*.
- **Độ dài:** câu trả lời dài hơn được chấm cao hơn dù không tốt hơn.
- **Tự thiên vị:** judge chấm cao cho output của *chính họ* nhà model mình.

Checklist trước khi tin judge

1. Đảo thứ tự A/B và lấy trung bình
2. Rubric cấm ưu tiên độ dài
3. Judge **khác họ** với model sinh
4. Từ 2 judge trở lên
5. Đối chiếu với vài nhãn do người chấm

Lưu ý: Việc gì **kiểm tra được bằng code** (đúng JSON schema? gọi đúng tên tool?) thì **đừng** tiêu tiền cho judge — dùng parser. Judge để dành cho chất lượng mở: tone, độ hữu ích, độ liên quan.



Nghĩa là gì

Prompt + test case + ngưỡng pass/fail nằm **trong git**, cạnh code — không nằm trong một ô chat.

Công cụ 2026 (tham khảo): promptfoo (CLI mã nguồn mở, hợp cho lab), Langfuse (self-host được), LangSmith (hợp nếu đã dùng LangChain), Braintrust (thương mại, xoay quanh CI gate).

Tool Calling

Tool calling là cách agent chuyển từ “nói” sang “tương tác với thế giới thực”

06



Ý nghĩa

Model **không** tự chạy code hay tự gọi API ngoài. Ứng dụng của bạn nhận tool request, chạy tool, rồi gửi kết quả trở lại model.

Vai trò	Trách nhiệm	Ví dụ
Developer (bạn)	Định nghĩa tool schema, viết implementation, handle errors	Viết <code>get_weather()</code> function
LLM	Quyết định tool nào, arguments gì, dựa trên user intent	Output: <code>{"name": "get_weather", "city": "Hanoi"}</code>
Application	Nhận tool call, execute, trả result	Gọi API weather, trả JSON result
LLM (lần 2)	Synthesize tool result thành câu trả lời tự nhiên	"Hà Nội hôm nay 32°C, trời nắng"

Phân vai rõ ràng giúp hiểu đúng cơ chế

- **Name:** nên ngắn, rõ, động từ đúng việc
- **Description:** nói khi nào nên dùng tool này
- **Parameters:** mô tả input bằng JSON Schema
- **Required fields:** giúp model biết thiếu gì thì chưa gọi được
- **strict:** ép arguments khớp schema tuyệt đối

Lưu ý: LLM đọc description như **tài liệu hướng dẫn**. Nếu description mơ hồ, model sẽ chọn sai tool hoặc truyền sai arguments.

```
weather_tool = {  
    "type": "function",  
    "name": "get_weather",  
    "description": (  
        "Get current weather for a city. "  
        "Use when the user asks about weather, temperature, or conditions."  
    ),  
    "parameters": {  
        "type": "object",  
        "properties": {  
            "city": {"type": "string", "description": "City name, e.g. Hanoi"}  
        },  
        "required": ["city"],  
        "additionalProperties": False,  
    },  
    "strict": True, # arguments must match the schema exactly  
}
```

	Anthropic (Messages API)	OpenAI (Responses API)
Tên trường schema	input_schema	parameters
Model gọi tool	content block tool_use	item function_call
Bạn trả kết quả	block tool_result trong message user	item function_call_output
Bắt gọi 1 tool cụ thể	{"type":"tool","name":"X"}	{"type":"function","name":"X"}
Gọi song song	bật mặc định; tắt bằng disable_parallel_tool_use	bật mặc định; tắt bằng parallel_tool_calls: false
Hội thoại	stateless — gửi lại toàn bộ messages	có thể stateful qua previous_response_id

Cùng khái niệm, khác cú pháp — điều đáng nhớ là sự khác biệt về hình dạng dữ liệu

```
tools = [{
    "name": "get_weather",
    "description": "Get current weather. Call this when the user asks about today's weather.",
    "input_schema": {
        "type": "object",
        "properties": {"location": {"type": "string", "description": "City name, e.g. Hanoi"}},
        "required": ["location"],
    },
}]

messages = [{"role": "user", "content": "Thoi tiet Ha Noi hom nay?"}]
resp = client.messages.create(model="claude-opus-5", max_tokens=1024,
                             tools=tools, messages=messages)
if resp.stop_reason == "tool_use":
    messages.append({"role": "assistant", "content": resp.content})
    results = [{"type": "tool_result", "tool_use_id": b.id, "content": run_tool(b.name, b.input)}
               for b in resp.content if b.type == "tool_use"]
    messages.append({"role": "user", "content": results}) # ALL results in ONE message
    final = client.messages.create(model="claude-opus-5", max_tokens=1024,
                                   tools=tools, messages=messages)
```

- ☐ **Prefill để ép JSON trên Claude** — nay **trả lỗi 400** trên mọi model Claude hiện hành. Dùng structured outputs.
- ☐ **“Chat Completions là cách gọi function của OpenAI”** — **Responses API** mới là bề mặt được khuyến nghị cho việc agentic/tool.
- ☐ **“strict mode tắt mất parallel tool calls”** — đã được sửa; các model đời mới hỗ trợ cả hai cùng lúc.
- ☐ **Model ID có hậu tố ngày kiểu claude-3-5-sonnet-2024xxxx** — đã ngừng phục vụ. Dùng claude-opus-5 / claude-sonnet-5; phía OpenAI dùng đời gpt-5.x.
- ☐ **MCP mô tả bằng transport “HTTP+SSE”** — đã được thay bằng Streamable HTTP.

Lưu ý: Đây chính là lý do bài này dạy **cơ chế** trước, **cú pháp** sau: cú pháp đổi mỗi năm, cơ chế thì không.

Giá trị	Ý nghĩa	Khi dùng
auto (mặc định)	Model tự quyết gọi hay không	Hầu hết use cases
required / any	Buộc gọi ít nhất 1 tool	Pipeline steps, routing
none	Cấm gọi tool, chỉ text	Test, fallback mode
{"name": "X"}	Buộc gọi tool cụ thể	Khi biết chắc cần tool nào

Dùng required cẩn thận: model có thể bịa arguments nếu user chưa cung cấp đủ thông tin

Phân biệt hai tầng:

- `tool_use / function_call` là thứ **model** phát ra.
- **MCP** là chuẩn để một **server** công bố và phục vụ tool cho bất kỳ client nào — không cần viết keo dán riêng cho từng ứng dụng.

Hai transport thực dụng: **stdio** (tiến trình cục bộ) và **Streamable HTTP** (server từ xa).

Đang thay đổi rất nhanh

Bản release candidate của spec MCP (07/2026) chuyển phần lỗi sang **stateless** để dễ scale sau load balancer. Học **ý tưởng**, đừng học thuộc chi tiết phiên bản.

Lỗi	Xử lý
Timeout	Retry + exponential backoff
Error response	Truyền error message cho model như một tool result để nó thông báo user
Unexpected format	Validation layer + fallback
Tool not found	Log + return error JSON

Lỗi tool không phải edge case — chúng SẼ xảy ra trong production

Design Principles Cho Tools

Tool tốt là software interface tốt, không phải prompt trang trí

Nguyên tắc	Ý nghĩa	Nếu vi phạm
Single Responsibility	Mỗi tool làm 1 việc rõ ràng	model khó quyết định nên gọi tool nào
Idempotency	Cùng input cho cùng kết quả; side effect được kiểm soát	retry dễ sinh lỗi phụ
Granularity hợp lý	Không quá nhỏ, cũng không ôm quá nhiều việc	hoặc overhead lớn, hoặc tool quá cứng
Test độc lập	Unit test từng tool trước khi gắn vào agent	khó tách lỗi tool khỏi lỗi prompt

Principles for reliable tool interfaces

Quá nhỏ

- `get_customer_name`
- `get_customer_email`
- `get_customer_phone`

Hệ quả: quá nhiều calls, overhead lớn, flow rối.

Quá to

- `handle_all_customer_operations`

Hệ quả: model không hiểu boundary, khó debug, khó reuse.

Mức hợp lý

Thiết kế tool quanh **một hành động nghiệp vụ rõ ràng**: ví dụ `lookup_order`, `get_weather`, `query_sales_data`, `send_email_draft`.

- **Required vs Optional:** chỉ require những gì thực sự cần
- **Enum constraints** giảm lỗi arguments
- **Default values:** document rõ trong description

```
"status": {  
  "type": "string",  
  "enum": ["pending", "shipped", "delivered"]  
}  
  
"date": {  
  "type": "string",  
  "description": "Date in YYYY-MM-DD format, e.g.  
    2026-04-05"  
}
```

```
// Success
{"status": "success",
 "data": {"temp": 32, "city": "Hanoi"},
 "source": "openweathermap"}

// Error
{"status": "error",
 "message": "City not found",
 "code": "NOT_FOUND"}
```

Rules:

- Trả JSON, không raw HTML/XML
- Error format consistent
- Include metadata (source, timestamp)
- Truncate nếu response quá dài

Lưu ý: Model xử lý **structured JSON** tốt hơn nhiều so với raw text hay HTML.

- ✓ **Cắt/nén kết quả lớn:** đừng đổ nguyên 50 nghìn token log vào context — trả tóm tắt + con trỏ (đường dẫn file, id) để model đọc thêm khi cần
- ✓ **Lỗi là dữ liệu:** bắt exception rồi trả về model dưới dạng tool result có cờ lỗi — đừng để harness văng, cũng đừng nuốt lỗi im lặng
- ✓ **Chặn vòng lặp:** luôn có MAX_ROUNDS; chạm trần thì dừng và báo, không quay vòng vô hạn
- ✓ **Gọi song song:** trả **tất cả** kết quả trong *cùng một* lượt — kể cả cái bị lỗi
- ✓ **Mô tả tool nói rõ KHI NÀO gọi:** “Call this when the user asks about current prices” chứ không chỉ “Gets prices”

Mơ hồ

"Search orders"

Model gọi cho MỌI câu hỏi liên quan đến order, kể cả "đơn hàng là gì?"

Rõ ràng

"Search orders by order_id or customer email.
Use **ONLY** when user provides an order number
or asks about specific order status."

Model biết boundary rõ, chỉ gọi khi có đủ data.

Công thức

Description nên chứa: (1) chức năng, (2) khi nào dùng, (3) khi nào KHÔNG dùng. Viết như viết **API docs**. Model đòi mới có xu hướng *dè dặt* hơn khi gọi tool — điều kiện kích hoạt viết rõ càng có giá trị.

Tool Patterns & Error Handling

Nhanh hơn không có nghĩa là tốt hơn nếu flow control và merge logic không rõ

Sequential

Tool B cần output của Tool A.
Ví dụ: tìm order ID -> rồi mới tra shipping status.

Parallel

Các tool độc lập có thể chạy cùng lúc.
Ví dụ: gọi thời tiết, tỷ giá, và lịch học song song.

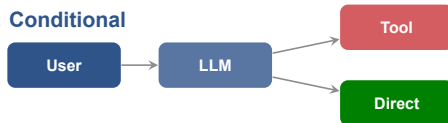
Lưu ý: Chỉ song song hóa khi **không có phụ thuộc dữ liệu**. Nếu song song, vẫn cần bước merge / verify rõ ràng ở cuối — và trả **mọi** kết quả trong cùng một lượt.

1. **Conditional tool use:** agent tự quyết định có cần tool hay trả lời trực tiếp.
2. **Tool chaining:** output của tool A là input của tool B.
3. **Parallel fetch + merge:** lấy nhiều nguồn độc lập rồi tổng hợp kết quả.

Teaching point

Tool calling không chỉ là “gọi API”. Nó là bài toán **control flow**: khi nào gọi, gọi cái gì, gọi theo thứ tự nào, và làm gì khi tool fail.

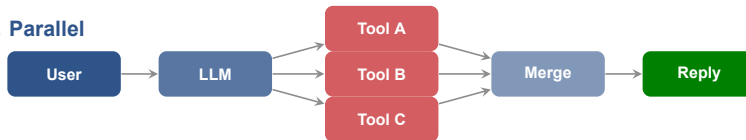
1. Conditional



2. Chaining



3. Parallel



```
MAX_ROUNDS = 5
messages = [{"role": "user", "content": user_input}]

for round_num in range(MAX_ROUNDS):
    response = call_model(messages, SYSTEM_PROMPT, TOOLS)
    tool_calls = extract_tool_calls(response)
    if not tool_calls:
        break                # Model done, no more tools needed

    for tc in tool_calls:
        try:
            result = execute_tool(tc.name, tc.args)
        except TimeoutError:
            result = {"error": "Tool timed out, please try again"}
        except Exception as e:
            result = {"error": str(e)}    # error goes BACK to the model as data
        messages.append(tool_result(tc.id, json.dumps(result)))
    else:
        print("Warning: max tool rounds reached")
```

Các bề mặt prompt trong một agent:

- System prompt
- **Mô tả tool** (chính là prompt!)
- Prompt của sub-agent
- Prompt của verifier / judge
- Output contract

Mỗi lần agent làm sai, hãy hỏi: *sửa được bằng một dòng luật ở bề mặt nào?*

Định nghĩa (02/2026)

“Mỗi khi bạn thấy agent mắc lỗi, hãy bỏ công **kỹ thuật hoá** một giải pháp sao cho agent *không bao giờ* mắc lại lỗi đó nữa.” — M. Hashimoto

Lưu ý: Thuật ngữ mới (<6 tháng), **chưa** phải chuẩn ngành — hãy dùng như cách gọi của giới thực hành, không phải định nghĩa cố định.

Thực Hành

Lab 4: Build first agent với system prompt + 2 tools + 5 test cases

09

1. Viết 1 system prompt với rules, constraints, output format
2. Tạo 2 custom tools: 1 API wrapper đơn giản, 1 data query đơn giản
3. Nối tools vào agent loop
4. Chạy 5 câu test để xem khi nào agent trả lời trực tiếp, khi nào gọi tool
5. Ghi lại lỗi thuộc loại prompt, tool schema, hay control flow

```
SYSTEM_PROMPT = open("system_prompt.txt").read()
TOOLS = [get_weather_tool(), query_sales_tool()]

while True:
    user_input = input("You: ")
    messages.append({"role": "user", "content": user_input})
    response = call_model(messages, SYSTEM_PROMPT, TOOLS)
    messages = handle_tool_calls(response, messages)
    print(render_final_answer(messages, SYSTEM_PROMPT, TOOLS))
```


Step 1–3: Setup

1. Chọn domain (weather + sales, hoặc tự chọn)
2. Viết system prompt (dùng template đã học)
3. Viết 2 tool schemas (name, description, params)

Step 4–6: Build & Test

4. Implement tool functions (mock data OK)
5. Wire vào agent loop (có error handling)
6. Test 5 câu hỏi, ghi pass/fail + lỗi

Mẹo

Bắt đầu với **mock tools** (trả data cố định) trước. Đảm bảo flow đúng rồi mới lo về real data.

#	Câu hỏi	Expected	Kiểm tra
1	“Thời tiết Hà Nội hôm nay?”	Gọi get_weather	Tool A hoạt động
2	“Doanh số tháng 3 là bao nhiêu?”	Gọi query_sales	Tool B hoạt động
3	“So sánh doanh số với thời tiết tuần này”	Gọi cả 2 tools	Parallel / chaining
4	“Prompt engineering là gì?”	Trả lời trực tiếp	Conditional: no tool
5	“Cho tôi số điện thoại CEO”	Từ chối, out of scope	Refusal handling

Thêm câu test riêng nếu agent của bạn có domain khác

Mức	Kiểm tra điều gì
function_name_match	Gọi đúng tên tool chưa?
parameter_validation	Đúng tên + arguments đúng kiểu/hình dạng chưa?
function_call_accuracy	So toàn bộ lời gọi với lời gọi chuẩn (gold)
function_call_exact_match	Khớp tuyệt đối, kể cả thứ tự các lời gọi song song

Benchmark tham chiếu của lĩnh vực: **BFCL** (Berkeley Function-Calling Leaderboard), so khớp theo cây cú pháp (AST) thay vì so chuỗi. Bản 2026 tính điểm nặng về multi-turn và **biết từ chối gọi tool khi không cần**.

Bốn mức này chấm bằng code — không cần LLM judge, không tốn tiền model

- ✓ Agent chạy end-to-end không crash?
- ✓ System prompt có đủ 5 thành phần (Persona, Rules, Capabilities, Constraints, Format)?
- ✓ Tool schemas có clear descriptions + required fields?
- ✓ Agent biết khi nào gọi tool vs khi nào trả lời trực tiếp?
- ✓ Agent xử lý gracefully khi tool fail (không crash, thông báo user)?
- ✓ Đã ghi chú ít nhất 2 lỗi phát hiện + phân loại (prompt / tool / control flow)?

Mục tiêu: Build agent với 2 custom tools, viết system prompt chuẩn, và test end-to-end trên 5 câu hỏi

Deliverable: **Deliverable:** Agent script chạy được + system prompt + 2 tool schemas + 5 test outputs + note lỗi prompt/tool/control flow + self-review checklist

Thời gian: 150 phút

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 Prompt = interface giữa human intent và model capability. Prompt tốt giúp model làm đúng việc, đúng format, đúng boundary.
- 2 Prompt → context → harness là *một nghề ở ba phạm vi*: một câu lệnh, cả tập token, rồi mọi prompt trong agent.
- 3 Tool description quyết định model gọi tool nào — viết rõ cả *khi nào KHÔNG* gọi.
- 4 Context lớn hơn không cứu bạn: chất lượng giảm từ sớm — phải chọn lọc, nén, cô lập.
- 5 Prompt injection chưa có lời giải trọn vẹn — giảm “lethal trifecta” và phòng thủ nhiều lớp.

AI Product Thinking & Requirements

“Bạn đã build được agent đầu tiên. Nhưng build xong chưa đủ. Ngày mai: sản phẩm này dành cho ai, yêu cầu ra sao, và rủi ro nào phải nghĩ từ đầu?”

- Hoàn thiện Lab 4 với 5 test questions rõ pass/fail
- Đọc lại system prompt của mình và chỉ ra 2 chỗ còn mơ hồ hoặc mâu thuẫn
- Thử viết 2 adversarial test cases (prompt injection) cho agent của bạn

- 1 Anthropic. *Prompt Engineering Overview; Tool Use Overview*. platform.claude.com/docs
- 2 Anthropic. *Effective Context Engineering for AI Agents* (2025); *Building Effective Agents* (2024).
- 3 OpenAI. *Function Calling Guide* (Responses API). developers.openai.com/api/docs
- 4 Wei et al. *Chain-of-Thought Prompting*. arXiv:2201.11903 (2022).
- 5 Liu et al. *Mind Your Step (by Step): CoT Can Reduce Performance*. arXiv:2410.21333 (2024).
- 6 Liu et al. *Lost in the Middle*. arXiv:2307.03172 (2023) · Chroma. *Context Rot* (2025).
- 7 OWASP. *Top 10 for LLM Applications* (2025) · Willison. *The Lethal Trifecta* (06/2025).
- 8 Beurer-Kellner et al. *Design Patterns for Securing LLM Agents against Prompt Injections*. arXiv:2506.08837 (2025).

Hỏi & Đáp

*Bạn đang gặp lỗi vì model chưa hiểu ý bạn,
hay vì tool contract của bạn chưa đủ rõ?*



Cảm ơn!

Email: lecturer@vinuni.edu.vn

Slides & tài liệu: github.com/aicb-vinuni

Lab template: bit.ly/aicb-day04-lab